

## Exercise 02: SQL FOUNDATIONS

### 01 The students Table

| student_id | name    | age | department |
|------------|---------|-----|------------|
| 1          | Alice   | 20  | IT         |
| 2          | Bob     | 22  | HR         |
| 3          | Charlie | 21  | IT         |
| 4          | Diana   | 23  | Finance    |
| 5          | Eve     | 22  | HR         |

01. List all departments in the student table.  
SQL query:

```
SELECT DISTINCT department  
FROM students;
```

Table output:

| department |
|------------|
| IT         |
| HR         |
| Finance    |

02: ~~Get average departments in the students table~~  
~~Expected columns: department, av~~

02. Get the average age of students per department.  
Expected columns: department, avg-age

SQL query:

```
SELECT department, AVG(age) AS avg-age  
FROM students  
GROUP BY department;
```

Table output:

| department | avg-age |
|------------|---------|
| IT         | 20.5    |
| HR         | 22.0    |
| Finance    | 23.0    |

03. Show departments with more than 1 student.  
Expected columns: department, student-count

SQL query:

```
SELECT department, COUNT(*) AS student-count  
FROM students  
GROUP BY departments  
HAVING COUNT(*) > 1;
```

Table output:

| department | student-count |
|------------|---------------|
| IT         | 2             |
| HR         | 2             |



04. Get all students whose age is between 21 and 23.

Expected columns: student-id, name, age, department

SQL query:

```
SELECT student-id,  
       name,  
       age,  
       department
```

FROM students

WHERE age BETWEEN 21 AND 23

Table output:

| Student-id | name    | age | department |
|------------|---------|-----|------------|
| 2          | Bob     | 22  | HR         |
| 3          | Charlie | 21  | IT         |
| 4          | Dianna  | 23  | Finance.   |
| 5          | Eve     | 22  | HR         |

05. List all students in the IT or HR department who are older than 21.

Expected columns: student-id, name, age, department

SQL query:

```
SELECT student-id,  
       name,  
       age,  
       department
```

FROM students



WHERE department IN ('IT', 'HR')  
AND age > 21;

Table output:

| Student_id | name | age | department |
|------------|------|-----|------------|
| 2          | Bob  | 22  | HR         |
| 5          | Eve  | 22  | HR         |

## 02 The Courses Table

| Course_id | Course_name    | department | credits |
|-----------|----------------|------------|---------|
| 101       | SQL Basics     | IT         | 3       |
| 102       | Python         | IT         | 4       |
| 103       | & Data Science | IT         | 4       |
| 104       | Excel          | Finance    | 2       |
| 105       | Statistics     | HR         | 3       |

06. Show total credits per department, only for departments with more than 5 total credits.

Expected Columns: department, total\_credits.

SQL query:

```
SELECT department, SUM(credits) AS total_credits
```

```
FROM courses
```

```
GROUP BY department
```

```
HAVING SUM(credits) > 5;
```



Table output:

| department | total_credits |
|------------|---------------|
| IT         | 11            |

07. List all courses that do not have 4 credits.  
Expected Columns: course\_id, course\_name, department, credits.

SQL query:

```
SELECT course_id,
       course_name,
       department,
       credits
FROM courses
WHERE NOT credits = 4;
```

Table Output:

| course_id | course_name | department | credits |
|-----------|-------------|------------|---------|
| 101       | SQL Basics  | IT         | 3       |
| 104       | Excel       | Finance    | 2       |
| 105       | Statistics  | HR         | 3       |

08. Show the top 3 courses by credits in descending order.

Expected columns: course\_id, course\_name, credits



SQL query :

```
SELECT course_id,  
       course_name,  
       credits
```

```
FROM Courses
```

```
ORDER BY credits DESC
```

```
LIMIT 3;
```

Table Output :

| Course_id | Course-name | Credits |
|-----------|-------------|---------|
| 102       | Python      | 4       |
| 103       | DataScience | 4       |
| 101       | SQL Basics  | 3       |

**Q3** The enrollments Table

| enrollment_id | Student-id | course-id | grade |
|---------------|------------|-----------|-------|
| 1             | 1          | 101       | 85    |
| 2             | 2          | 102       | 78    |
| 3             | 3          | 103       | 90    |
| 4             | 4          | 104       | 88    |
| 5             | 5          | 105       | 82    |

Q9. Get the maximum, minimum, and average grade across all enrollments

Expected columns: max-grade, min-grade, avg-grade



SQL query:

```
SELECT MAX(grade) AS max-grade,  
       MIN(grade) AS min-grade,  
       AVG(grade) AS avg-grade  
FROM enrollments;
```

Table output :

| max-grade | min-grade | avg-grade |
|-----------|-----------|-----------|
| 90        | 78        | 84.6      |

10. Count how many enrollments exist per course.  
Expected columns: course\_id, enrollment\_count.

SQL query:

```
SELECT course_id,  
       COUNT(*) AS enrollment-count  
FROM enrollments  
GROUP BY course_id;
```

Table output :

| course_id | enrollment_count |
|-----------|------------------|
| 101       | 1                |
| 102       | 1                |
| 103       | 1                |
| 104       | 1                |
| 105       | 1                |



#### 04 The salaries table.

| employee-id | name  | department | salary | bonus |
|-------------|-------|------------|--------|-------|
| 1           | Tom   | IT         | 60000  | 5000  |
| 2           | Jerry | HR         | 55000  | 4000  |
| 3           | Spike | Finance    | 70000  | 6000  |
| 4           | TJke  | IT         | 62000  | 5500  |
| 5           | Butch | HR         | 54000  | 3500  |

11. Find total salary and total bonus per department.

Expected columns: department, total\_salary, total\_bonus.

SQL query:

```
SELECT department,  
       sum(salary) AS total_salary,  
       sum(bonus) AS total_bonus
```

FROM salaries

GROUP BY department;

Table output:

| department | total_salary | total_bonus |
|------------|--------------|-------------|
| IT         | 122000       | 10500       |
| HR         | 109000       | 7500        |
| Finance    | 70000        | 6000        |



12. Show departments where average salary is greater than 55,000.

Expected Columns: department, avg-salary.

SQL query:

```
SELECT department,  
       AVG(salary) AS avg-salary  
FROM salaries  
GROUP BY department  
HAVING AVG(salary) > 55000;
```

Table Output:

| department | avg salary |
|------------|------------|
| IT         | 61000      |
| Finance    | 70000      |

13. List employees whose salary plus bonus is greater than 60,000

Expected Columns: employee-id, name, salary, bonus, total compensation.

SQL query:

```
SELECT employee_id,  
       name,  
       salary,  
       bonus,  
       (salary + bonus) AS total-compensation  
FROM salaries  
WHERE (salary + bonus) > 60000;
```



Table output:

| employee-id | name  | salary | bonus | total-compensation |
|-------------|-------|--------|-------|--------------------|
| 1           | Tom   | 60000  | 5000  | 65000              |
| 3           | Spike | 70000  | 6000  | 76000              |
| 4           | Tyke  | 62000  | 5500  | 67500              |

**Q5** The projects Table

| project-id | project-name   | department | budget |
|------------|----------------|------------|--------|
| 1          | AI App         | IT         | 120000 |
| 2          | Payroll System | Finance    | 80000  |
| 3          | Dashboard      | IT         | 150000 |
| 4          | Website        | Marketing  | 60000  |
| 5          | HR Portal      | HR         | 50000  |

14. Show total and average budget per department.  
Only include departments with average budget above 70,000.

Expected columns: department, total-budget, avg-budget

SQL query:

```
SELECT department,
       SUM(budget) AS total-budget,
       AVG(budget) AS avg-budget
```

```
FROM projects
```

```
GROUP BY department
```

```
HAVING AVG(budget) > 70000;
```



Table output :

| department | total_budget | avg_budget |
|------------|--------------|------------|
| IT         | 270000       | 135000     |
| Finance    | 80000        | 80000      |

15. List all projects with budgets between 50,000 and 120,000, excluding the Marketing department.

Expected columns: project-id, project-name, department, budget.

SQL query:

```
SELECT project-id,  
       project-name,  
       department,  
       budget
```

```
FROM projects
```

```
WHERE budget BETWEEN 50000 AND 120000  
      AND NOT department = 'Marketing';
```

Table output :

| project-id | project-name  | department | budget |
|------------|---------------|------------|--------|
| 1          | AI App        | IT         | 120000 |
| 2          | Payrol System | Finance    | 80000  |
| 5          | HR Portal     | HR         | 50000  |